

PROGRAMACIÓN DE MICROCONTROLADORES

REPASO DE C - segunda parte

**Carrera de Especialización en Sistemas Embebidos
Facultad de Ingeniería - UBA**

Variables

- **Locales:**

- Se declaran dentro de una función.
- Solo son accesibles por la función donde fueron declaradas.
- Se alojan en memoria en registros o la pila de una función.

- **Globales:**

- Se declaran fuera de toda función.
- Pueden ser accedidas desde cualquier parte del programa.
- Se alojan en direcciones fijas de RAM.

Las locales tienen prioridad ante una global con el mismo nombre

Storage Classes en C

Las clases de almacenamiento en C determinan:

- **Alcance** (scope), define la visibilidad de la variable en el programa (dónde puede ser accedida).
- **Duración** (lifetime), indica cuánto tiempo existe la variable en la memoria.
- **Vinculación** (Linkage)
 - **None**: es local a su bloque.
 - **Internal**: es visible sólo dentro del mismo archivo (usando `static`).
 - **External**: puede ser referenciada desde otros archivos (usando `extern`).

Storage Classes en C

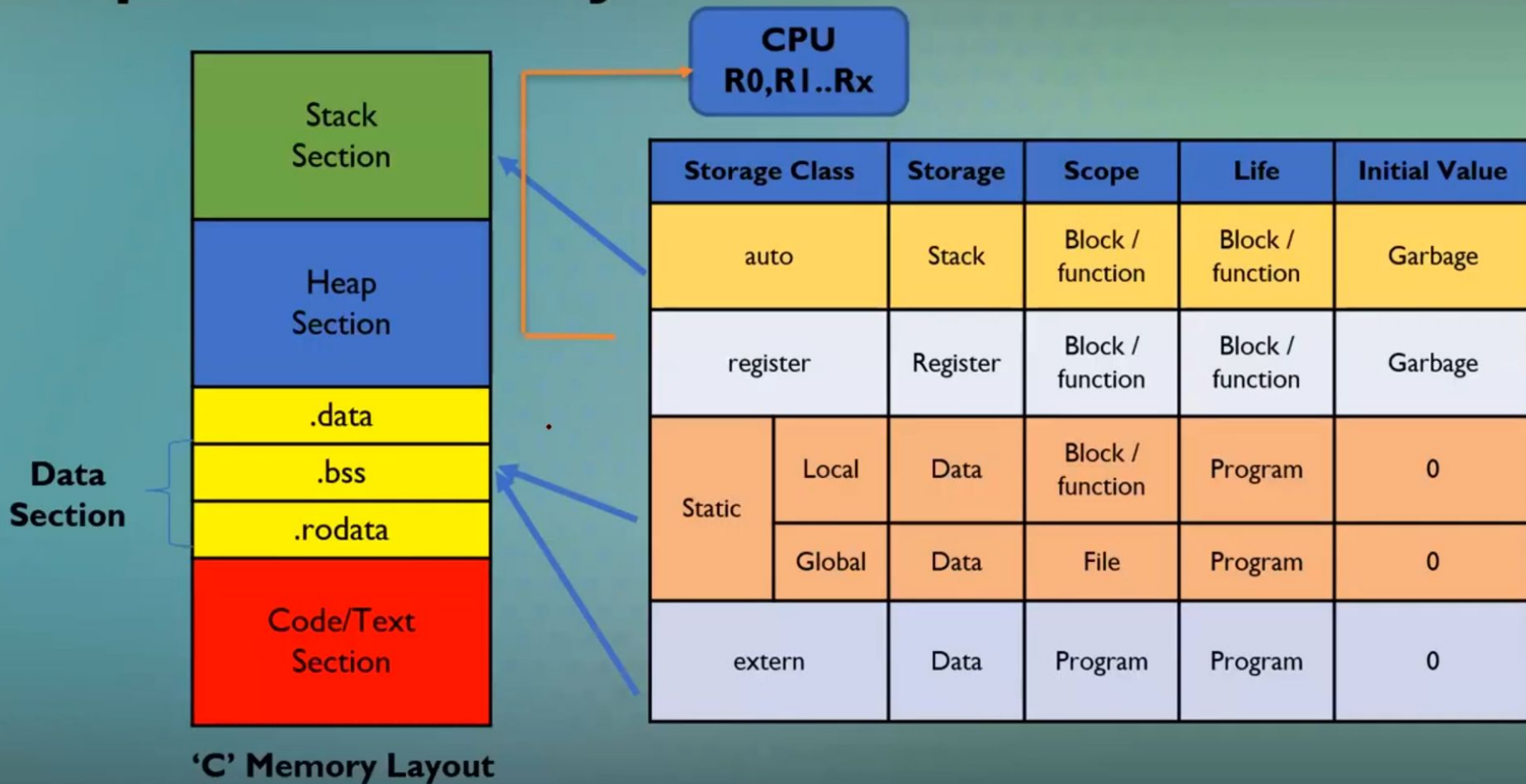
- **Auto:** es la clase de almacenamiento predeterminada para las **variables locales**. El compilador asume que todas las variables locales son *auto* por defecto.
- **Static:**
 - Una variable global estática se convierte en **privada** y su visibilidad se reduce al archivo .c donde haya sido declarada.
 - Una variable local estática posee una **dirección de memoria fija** y conserva su valor entre sucesivas llamadas a la función.
- **Register:** sugiere al compilador que la variable local sea colocada en algún registro CPU para optimizar su acceso.
- **Extern:** indica que la variable ya está definida en otro archivo.

C memory layout

Cómo se organiza la memoria de un programa durante su ejecución

- **Text Segment (Código):** Contiene el código ejecutable. Es de solo lectura.
- **Data Segment (Datos Estáticos):**
 - **Initialized Data Segment:** Variables globales y estáticas inicializadas.
 - **Uninitialized Data Segment (BSS):** Variables globales y estáticas no inicializadas.
- **Heap:** Memoria para la asignación dinámica, crece hacia arriba.
- **Stack:** Gestiona variables locales y control de funciones, crece hacia abajo.

Important 'C' Keywords



Variables - modificadores de tipo

- **Const:**

- La variable no puede modificarse una vez inicializada.
- Se almacena en ROM

- **Volatile:**

- Indica que el compilador que no debe aplicar optimizaciones al código ya que una variable volatile puede cambiar su valor por una acción externa al software:
 - Evento de hardware, periféricos, etc.
 - Rutina de interrupción.

- **Restrict:**

- Se usa con punteros y sugiere que el puntero es el único medio de acceso al objeto apuntado, lo que permite optimizaciones.

- **Otros:**

- signed/unsigned
- short/long

Ejemplo de código

```
#include <stdio.h>
#include <stdlib.h>

int global_var;           // BSS segment
const int const_var = 10; // Data segment (readonly)
static int static_var = 5; // Data segment (initialized)

void function() {
    auto int local_var = 3; // Stack
    register int reg_var = 2; // Stack (CPU register)
    static int local_static = 4; // Data segment (initialized)
    int *dynamic_var = (int *)malloc(sizeof(int)); // Heap
}

int main() {
    function();
    return 0;
}
```

Power of 10 rules ([link](#))

1. Evitar control de flujo complejo como **goto** y **recursiones**.
2. Todos los bucles deben tener límites fijos.
3. Evitar el uso de asignación dinámica de memoria.
4. Restringir las funciones a una página impresa simple.
5. Usar un mínimo de dos chequeos en tiempo de ejecución por función (assertions).
En general: pre y post condiciones.
6. Restringir la visibilidad de la información al mínimo posible.
7. Chequear el valor de retorno de todas las funciones *non-void*.
8. Usar el preprocesador moderadamente.
9. Limitar el uso de punteros a una sola desreferencia.
No usar punteros a funciones.
10. Compilar con todos los warnings activos.

Repaso

- `static` (storage class)

Usar agresivamente `static` para encapsular la información.

- `const` (type qualifier)

Protección contra modificaciones en tiempo de ejecución.

- `volatile` (type qualifier)

Para datos que cambian fuera del flujo normal del programa